

Getting Started with Pelee and Rondeau

Fluids Lab

September 22, 2026

Contents

1	Introduction	5
1.1	Why Are We Using Servers?	5
1.2	Important Concepts	5
1.3	Learning Objectives	5
2	Connecting to a Server	6
2.1	What is SSH?	6
2.2	Windows	6
2.3	macOS and Linux	6
2.4	First Login	6
2.5	Successful Login	6
2.6	Logging Out	6
2.7	Exercise	6
3	Linux Navigation	7
3.1	Where Am I?	7
3.2	Listing Files	7
3.3	Changing Directories	7
3.4	Creating Directories	7
3.5	Useful Shortcuts	7
3.6	Exercise	7
4	Files and Directories	8
4.1	Creating Files	8
4.2	Copying Files	8
4.3	Moving and Renaming Files	8
4.4	Deleting Files	8
4.5	Working with Directories	8
4.6	Exercise	8
5	Editing Files	9
5.1	Using Nano	9
5.2	Saving Changes	9
5.3	Exiting	9
5.4	Viewing a File	9
5.5	Exercise	9

6	Running Programs	10
6.1	Python	10
6.2	Julia	10
6.3	Useful Commands	10
7	Conda Environments	11
7.1	Creating an Environment	11
7.2	Activating an Environment	11
7.3	Installing Packages	11
7.4	Leaving an Environment	11
8	Python Virtual Environments	12
8.1	Creating a Virtual Environment	12
8.2	Activating the Environment	12
8.3	Installing Packages	12
8.4	Leaving the Environment	12
8.5	When Should I Use Conda?	12
8.6	When Should I Use a Virtual Environment?	13
8.7	Exercise	13
8.8	Recommended Practice	13
9	Long-Running Jobs	14
9.1	Using tmux	14
9.2	Detach	14
9.3	Reconnect	14
9.4	Exercise	14
10	Monitoring Resources	15
10.1	CPU and Memory	15
10.2	Disk Usage	15
10.3	Exercise	15
11	GPU Computing	16
11.1	Checking GPU Usage	16
11.2	Good Practice	16
11.3	Exercise	16
12	Transferring Files	17
12.1	SCP	17
12.2	RSYNC	17
12.3	Exercise	17
13	Git Basics	18
13.1	Cloning a Repository	18
13.2	Saving Changes	18
13.3	Uploading Changes	18
13.4	Getting Updates	18
13.5	Exercise	18

14 Good Computing Citizenship	19
14.1 Storage	19
14.2 File Naming	19
15 Reproducible Research	20
15.1 Recommended Project Structure	20
15.2 Best Practices	20
15.3 Exercise	20
16 Troubleshooting	21
16.1 Command Not Found	21
16.2 Disk Full	21
16.3 Permission Denied	21
16.4 Program Crashed	21
16.5 Before Asking for Help	21
A Vim Survival Guide	22
A.1 Insert Mode	22
A.2 Return to Command Mode	22
A.3 Common Commands	22
A.4 Most Important Command	22
B File Permissions	22
B.1 Making a Script Executable	23
B.2 Useful Commands	23
C SSH Keys	24
C.1 Create a Key Pair	24
C.2 Install the Public Key	24
C.3 Test	24
D VSCode Remote SSH	25
D.1 Install	25
D.2 Connect	25
D.3 Benefits	25
E Jupyter Notebooks	26
E.1 Start Jupyter	26
E.2 Create an SSH Tunnel	26
E.3 Open in a Browser	26
F Alliance Clusters	27
G Pelee and Rondeau Cheat Sheet	28
H Multi-GPU Computing with Julia	29
H.1 What is CUDA-Aware MPI?	29
H.2 Building CUDA-Aware OpenMPI	29
H.3 Installation	29

H.4	Updating Your Environment	30
H.5	Configuring MPI.jl	30
H.6	Verifying GPU Assignment	30
H.7	Selecting GPUs	31
H.8	Troubleshooting	31
I	Multi-GPU Computing with Python	32
I.1	Checking Available GPUs	32
I.2	Selecting GPUs	32
I.3	MPI with Python	32
I.4	Multi-GPU PyTorch	33
I.5	Troubleshooting	33

1 Introduction

1.1 Why Are We Using Servers?

Most scientific models and datasets are too large to run efficiently on a laptop. The Fluids Lab servers, **Pelee** and **Rondeau**, provide access to faster CPUs, GPUs, larger memory, and shared storage.

You may think of them as powerful Linux computers that you access remotely.

Your Laptop $\longleftrightarrow$ Pelee / Rondeau

You work from your laptop, but the calculations happen on the server.

1.2 Important Concepts

Multi-user systems. Many researchers share the same server.

CPU versus GPU. A CPU is designed for general-purpose computing. A GPU is designed for highly parallel computations.

Home directory. Every user has a home directory:

`/home/username`

Note: for historical reasons at UWaterloo Math, `/home` is an alias for `/u`, although they have the same purpose and meaning.

Persistence. Files remain on the server after you log out.

1.3 Learning Objectives

- connect to Pelee and Rondeau;
- navigate the Linux filesystem;
- transfer files;
- run Python and Julia programs;
- monitor resource usage.

2 Connecting to a Server

2.1 What is SSH?

SSH (Secure Shell) is the standard method for connecting to a remote Linux machine.

```
ssh username@pelee.math.private.uwaterloo.ca
```

```
ssh username@rondeau.math.private.uwaterloo.ca
```

2.2 Windows

Open Windows Terminal and run the SSH command.

2.3 macOS and Linux

Open a terminal and run the SSH command.

Note: Nothing appears on the screen while typing your password. This is normal.

2.4 First Login

Type **yes** when asked whether you trust the host.

2.5 Successful Login

```
username@pelee:~$
```

2.6 Logging Out

```
exit
```

2.7 Exercise

1. Connect to Pelee.
2. Run `pwd`.
3. Disconnect using `exit`.

3 Linux Navigation

Most interactions with a Linux server occur through a command-line interface.

3.1 Where Am I?

```
pwd
```

Example:

```
/home/username
```

3.2 Listing Files

```
ls
ls -l
ls -lh
```

3.3 Changing Directories

```
cd project1
cd ..
cd
```

Typing `cd` without an argument returns you to your home directory.

3.4 Creating Directories

```
mkdir project1
mkdir code data figures
```

3.5 Useful Shortcuts

Symbol	Meaning
.	Current directory
..	Parent directory
~	Home directory

3.6 Exercise

Create:

Try the following commands:

```
mkdir research
cd research
mkdir code data figures
ls
```

4 Files and Directories

Files and directories are the basic building blocks of a Linux filesystem.

4.1 Creating Files

Create an empty file:

```
touch notes.txt
```

Verify that it exists:

```
ls
```

4.2 Copying Files

Copy a file:

```
cp notes.txt backup.txt
```

The original file is unchanged.

4.3 Moving and Renaming Files

Rename a file:

```
mv notes.txt notes_old.txt
```

Move a file into another directory:

```
mv notes_old.txt data/
```

4.4 Deleting Files

Delete a file:

```
rm notes.txt
```

Warning: Deleted files cannot usually be recovered.

4.5 Working with Directories

Copy a directory and its contents:

```
cp -r project project_backup
```

Delete a directory and its contents:

```
rm -r project_backup
```

4.6 Exercise

Create a file called `notes.txt`. Copy it, rename it, and then delete the copy.

5 Editing Files

Many scientific workflows require editing scripts, configuration files, and text files.

5.1 Using Nano

Nano is a simple text editor that is available on most Linux systems.

Open a file:

```
nano notes.txt
```

If the file does not exist, it will be created.

5.2 Saving Changes

To save:

```
Ctrl+O
```

Press Enter to confirm.

5.3 Exiting

To exit:

```
Ctrl+X
```

5.4 Viewing a File

Display a file:

```
cat notes.txt
```

For longer files:

```
less notes.txt
```

Press q to quit.

5.5 Exercise

Create a file called `todo.txt`. Add three lines of text using Nano.

6 Running Programs

Most users connect to Pelee and Rondeau in order to run scientific software.

6.1 Python

Example file, `hello.py`, with contents:

```
print("Hello World")
```

Run using:

```
python hello.py
```

6.2 Julia

Example file, `hello.jl`, with contents:

```
println("Hello World")
```

Run using:

```
julia hello.jl
```

6.3 Useful Commands

```
which python  
echo $PATH
```

7 Conda Environments

Different projects often require different software versions.

7.1 Creating an Environment

```
conda create -n ocean python=3.12
```

7.2 Activating an Environment

```
conda activate ocean
```

7.3 Installing Packages

```
conda install numpy
```

7.4 Leaving an Environment

```
conda deactivate
```

8 Python Virtual Environments

Python includes a built-in mechanism for creating isolated environments.

Virtual environments provide many of the same benefits as Conda and are often simpler for Python-only projects.

8.1 Creating a Virtual Environment

Create a new environment called `venv`:

```
python -m venv venv
```

This creates a directory containing an isolated Python installation.

8.2 Activating the Environment

Activate the environment:

```
source venv/bin/activate
```

Your command prompt should change to indicate that the environment is active.

8.3 Installing Packages

Use `pip` to install packages:

```
pip install numpy
```

You may verify the installation:

```
python -c "import numpy"
```

8.4 Leaving the Environment

Deactivate the environment:

```
deactivate
```

8.5 When Should I Use Conda?

Conda is often easier when working with:

- JupyterLab
- CUDA-enabled Python packages
- packages with compiled dependencies
- mixed Python and non-Python software

8.6 When Should I Use a Virtual Environment?

Virtual environments are often a good choice for:

- lightweight Python projects
- software development
- projects using only pip-installed packages

8.7 Exercise

Create a virtual environment, install NumPy, verify that it imports successfully, and then deactivate the environment.

One thing I would add

Since your students use HPC systems, add this box at the end:

8.8 Recommended Practice

Create a separate environment for each project.

For example:

```
arctic_project_env  
oceananigans_env  
machine_learning_env
```

Avoid installing large numbers of unrelated packages into a single environment.

9 Long-Running Jobs

Many computations run for hours or days. If your SSH connection is interrupted, the job may stop.

9.1 Using tmux

Start a tmux session:

```
tmux
```

Run your program normally.

9.2 Detach

Leave the session running:

```
Ctrl+b d
```

9.3 Reconnect

List sessions:

```
tmux ls
```

Reconnect:

```
tmux attach
```

9.4 Exercise

Start tmux, run a simple program, detach, and reconnect.

10 Monitoring Resources

Before running large jobs, monitor resource usage.

10.1 CPU and Memory

```
htop
```

or

```
top
```

10.2 Disk Usage

Available space:

```
df -h
```

Directory sizes:

```
du -sh *
```

10.3 Exercise

Determine how much free disk space is available and identify the largest directory in your project.

11 GPU Computing

Many scientific applications can use GPUs.

11.1 Checking GPU Usage

```
nvidia-smi
```

This displays:

- GPU memory usage;
- GPU utilization;
- running processes.

11.2 Good Practice

Always check whether a GPU is busy before starting large jobs.

11.3 Exercise

Run `nvidia-smi` and identify the available GPUs.

12 Transferring Files

Move files between your laptop and the server using SCP or rsync.

12.1 SCP

```
scp file.txt username@pelee.math.private.uwaterloo.ca:~
```

12.2 RSYNC

```
rsync -av data/ username@pelee.math.private.uwaterloo.ca:data/
```

12.3 Exercise

Transfer a small text file and verify the transfer using `ls`.

13 Git Basics

Git is a version control system that helps track changes to code and documents.

13.1 Cloning a Repository

```
git clone https://github.com/user/repo.git
```

13.2 Saving Changes

```
git add file.py  
git commit -m "Update analysis"
```

13.3 Uploading Changes

```
git push
```

13.4 Getting Updates

```
git pull
```

13.5 Exercise

Create a repository, modify a file, commit the change, and push it.

14 Good Computing Citizenship

Pelee and Rondeau are shared resources.

- Keep directories organized.
- Delete unnecessary files.
- Do not monopolize CPUs or GPUs.
- Use descriptive file names.
- Document your work.

14.1 Storage

Home directories should only be used for storing small files like text files or code. Large data, such as binary output datasets, should be stored on the large RAID (Redundant Array of Inexpensive Disks) volumes, named 'fsys1' and 'fsys2'. Your folder on these storage volumes can be accessed on either pelee or rondeau by `cd`'ing to:

```
/mnt/autofs/sutton.math/fsys1/username  
/mnt/autofs/sutton.math/fsys2/username
```

Replace 'username' in the above with your UWaterloo 8-character userid.

14.2 File Naming

Avoid:

```
test  
test2  
test_final  
test_final2
```

Prefer:

```
arctic_run_2026_09_18
```

15 Reproducible Research

Good research should be easy for others (and your future self) to reproduce.

15.1 Recommended Project Structure

```
project/  
    code/  
    data/  
    figures/  
    notebooks/  
    manuscript/
```

15.2 Best Practices

- Use Git.
- Use Conda environments.
- Keep raw data separate from results.
- Write a README file.

15.3 Exercise

Create a project directory using the structure above.

16 Troubleshooting

16.1 Command Not Found

```
which python
```

Verify that the program is installed and available in your PATH.

16.2 Disk Full

```
df -h
```

16.3 Permission Denied

```
ls -l
```

Check file ownership and permissions.

16.4 Program Crashed

Check:

- available memory;
- available disk space;
- error messages.

16.5 Before Asking for Help

Include:

- the command you ran;
- the full error message;
- what you have already tried.

A Vim Survival Guide

Sooner or later you will encounter Vim.

Open a file:

```
vim file.txt
```

A.1 Insert Mode

Press

```
i
```

to begin editing.

A.2 Return to Command Mode

Press

```
Esc
```

A.3 Common Commands

Save:

```
:w
```

Quit:

```
:q
```

Save and quit:

```
:wq
```

Quit without saving:

```
:q!
```

A.4 Most Important Command

If you become stuck:

```
Esc :q!
```

B File Permissions

Linux controls who may read, write, and execute files.

Display permissions:

```
ls -l
```

Example:

```
-rwxr-xr-x script.sh
```

B.1 Making a Script Executable

```
chmod u+x script.sh
```

You can then run

```
./script.sh
```

B.2 Useful Commands

Give read and write permission to the owner:

```
chmod 600 file.txt
```

Make a file executable:

```
chmod 755 script.sh
```

C SSH Keys

SSH keys allow password-free login.

C.1 Create a Key Pair

```
ssh-keygen
```

Accept the default location.

C.2 Install the Public Key

```
ssh-copy-id username@pelee.math.private.uwaterloo.ca
```

C.3 Test

```
ssh username@pelee.math.private.uwaterloo.ca
```

You should no longer be asked for a password.

D VSCode Remote SSH

Many users prefer VSCode instead of terminal editors.

D.1 Install

Install

- Visual Studio Code
- Remote SSH Extension

D.2 Connect

Open the command palette:

Ctrl+Shift+P

Select

Remote-SSH: Connect to Host

and connect to Pelee or Rondeau.

D.3 Benefits

- graphical file browser;
- interactive terminal;
- syntax highlighting;
- Git integration.

E Jupyter Notebooks

Jupyter allows interactive computing through a web browser.

E.1 Start Jupyter

```
jupyter lab --no-browser
```

The server will display a URL and port number.

E.2 Create an SSH Tunnel

From your laptop:

```
ssh -L 8888:localhost:8888 username@pelee
```

E.3 Open in a Browser

Open

```
http://localhost:8888
```

in your browser.

F Alliance Clusters

The skills learned on Pelee and Rondeau transfer directly to national HPC systems.

Examples:

- Narval
- Béluga
- Cedar
- Graham

Daily tasks remain similar:

```
ssh cd ls tmux git conda
```

The major difference is that most large jobs must be submitted through a scheduler.

G Pelee and Rondeau Cheat Sheet

Task	Command
Login	ssh username@pelee
Current directory	pwd
List files	ls -lh
Create directory	mkdir project
Edit file	nano file.txt
Copy file	cp old new
Move file	mv old new
Delete file	rm file.txt
Start tmux	tmux
Reconnect tmux	tmux attach
Disk usage	df -h
Directory size	du -sh *
Check GPUs	nvidia-smi
Logout	exit

H Multi-GPU Computing with Julia

Many Julia applications can use multiple GPUs simultaneously. Typical examples include:

- Oceananigans.jl
- MPI-based PDE solvers
- Distributed machine learning
- Large ensemble simulations

Multi-GPU programs typically require a CUDA-aware MPI installation.
The typical workflow is:

1. Build a CUDA-aware MPI library.
2. Configure MPI.jl to use that library.
3. Launch Julia using `mpiexec`.
4. Verify that each MPI rank can access a GPU.

H.1 What is CUDA-Aware MPI?

CUDA-aware MPI allows GPU memory to be communicated directly between GPUs without first copying data through host memory.

This can significantly improve performance for multi-GPU applications.

H.2 Building CUDA-Aware OpenMPI

Download OpenMPI:

```
curl -fLO \
https://download.open-mpi.org/release/open-mpi/v4.1/openmpi-4.1.7.tar.bz2
```

Extract:

```
tar xvf openmpi-4.1.7.tar.bz2
```

Enter the source directory:

```
cd openmpi-4.1.7
```

H.3 Installation

Most users will not have administrator privileges on shared servers. Instead, install OpenMPI in your home directory.

Configure OpenMPI:

```
./configure \
--prefix=$HOME/openmpi-4.1.7-with-cuda \
--with-cuda=/opt/uw/cuda/12.3.2
```

Compile:

```
make -j8
```

Install:

```
make install
```

H.4 Updating Your Environment

Add the following lines to your `~/.bashrc` file:

```
export PATH=$HOME/openmpi-4.1.7-with-cuda/bin:$PATH
```

```
export LD_LIBRARY_PATH=$HOME/openmpi-4.1.7-with-cuda/lib:$LD_LIBRARY_PATH
```

Reload your shell:

```
source ~/.bashrc
```

Verify:

```
which mpiexec
```

Expected output:

```
/home/username/openmpi-4.1.7-with-cuda/bin/mpiexec
```

Verify CUDA:

```
nvidia-smi
```

H.5 Configuring MPI.jl

Inside Julia:

```
using MPIPreferences
```

```
MPIPreferences.use_system_binary()
```

Restart Julia and verify:

```
using MPI
MPI.versioninfo()
```

H.6 Verifying GPU Assignment

Create a file called `test.jl`:

```
using MPI
using CUDA

MPI.Init()

rank = MPI.Comm_rank(MPI.COMM_WORLD)

println( "Rank $rank using GPU $(CUDA.device())" )

MPI.Finalize()
```

Launch four MPI ranks:

```
mpiexec -n 4 julia test.jl
```

Expected output:

```
Rank 0 using GPU CuDevice(0)
Rank 1 using GPU CuDevice(1)
Rank 2 using GPU CuDevice(2)
Rank 3 using GPU CuDevice(3)
```

H.7 Selecting GPUs

You can restrict a program to specific GPUs using `CUDA_VISIBLE_DEVICES`.

For example, to use only GPU 0:

```
CUDA_VISIBLE_DEVICES=0 julia script.jl
```

To use GPUs 1 and 2:

```
CUDA_VISIBLE_DEVICES=1,2 julia script.jl
```

This is useful on shared servers where multiple users may be using GPUs simultaneously.

H.8 Troubleshooting

If `MPI.jl` does not appear to use your custom OpenMPI installation, check

```
using MPI
MPI.versioninfo()
```

and verify that the reported MPI library corresponds to the CUDA-aware OpenMPI installation.

If `MPI.jl` does not appear to use the correct MPI library, rerun the commands in the previous section and verify that the reported MPI installation corresponds to your CUDA-aware OpenMPI installation.

I Multi-GPU Computing with Python

Many Python applications can use multiple GPUs simultaneously.

Typical examples include:

- PyTorch
- TensorFlow
- JAX
- MPI-based scientific computing

I.1 Checking Available GPUs

Before running a multi-GPU application, verify that Python can see the GPUs.

Create a file called `testgpu.py`:

```
import torch

print(torch.cuda.device_count())
```

Run:

```
python test_gpu.py
```

Expected output:

```
4
```

if four GPUs are available.

I.2 Selecting GPUs

A program can be restricted to specific GPUs using `CUDA_VISIBLE_DEVICES`.

Use only GPU 0:

```
CUDA_VISIBLE_DEVICES=0 python script.py
```

Use GPUs 1 and 2:

```
CUDA_VISIBLE_DEVICES=1,2 python script.py
```

This is useful on shared servers where multiple users may be using GPUs simultaneously.

I.3 MPI with Python

Many scientific Python applications use MPI through `mpi4py`.

Install:

```
pip install mpi4py
```

Create a file called `testmpi.py`:

```
from mpi4py import MPI

comm = MPI.COMM_WORLD

print( f"Rank {comm.rank} " f"of {comm.size}" )
```

Run:

```
mpiexec -n 4 python test_mpi.py
```

Expected output:

```
Rank 0 of 4
Rank 1 of 4
Rank 2 of 4
Rank 3 of 4
```

I.4 Multi-GPU PyTorch

PyTorch supports multiple GPUs using Distributed Data Parallel (DDP).

Check visible GPUs:

```
import torch

print(torch.cuda.device_count())
```

A typical launch command is:

```
torchrun \
--nproc_per_node=4 \
train.py
```

where one process is launched for each GPU.

I.5 Troubleshooting

Verify that Python can see the GPUs:

```
nvidia-smi
```

Verify that PyTorch detects CUDA:

```
python -c \
"import torch; print(torch.cuda.is_available())"
```

Expected output:

```
True
```